

INFORME TÉCNICO DE ARQUITECTURA DE SOFTWARE Y DOCUMENTACIÓN DEL PROYECTO

Gestor Inteligente de Clientes (GIC)

DATOS GENERALES DEL PROYECTO

Parámetro	Detalle	Parámetro	Detalle
Proyecto:	Gestor inteligente de clientes (GIC) - Módulo 4	Fecha:	Agosto 2026
Lenguaje/GUI:	Python 3.14 / Tkinter	Persistencia:	SQLite3 / JSON / CSV / TXT
Estado de Tests:	8/8 PASSED (OK)	Patrón de Diseño:	Arquitectura Modular Multicapa POO

```

PS C:\Users\Conny\Desktop\Python\Proyectos\proyecto_modulo4> & C:\Users\Conny\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Conny/Desktop/Python/Proyectos/proyecto_modulo4/main.py
PS C:\Users\Conny\Desktop\Python\Proyectos\proyecto_modulo4> python -m unittest discover -s test
.....[API Externo] Conexión exitosa. Email 'user1@gmail.com' validado.
[API Notificación] Email de bienvenida enviado a user1@gmail.com.
[API Externo] Conexión exitosa. Email 'user2@gmail.com' validado.
.[API Externo] Conexión exitosa. Email 'test@gmail.com' validado.
[API Notificación] Email de bienvenida enviado a test@gmail.com.
..
-----
Ran 8 tests in 1.698s

OK
PS C:\Users\Conny\Desktop\Python\Proyectos\proyecto_modulo4>

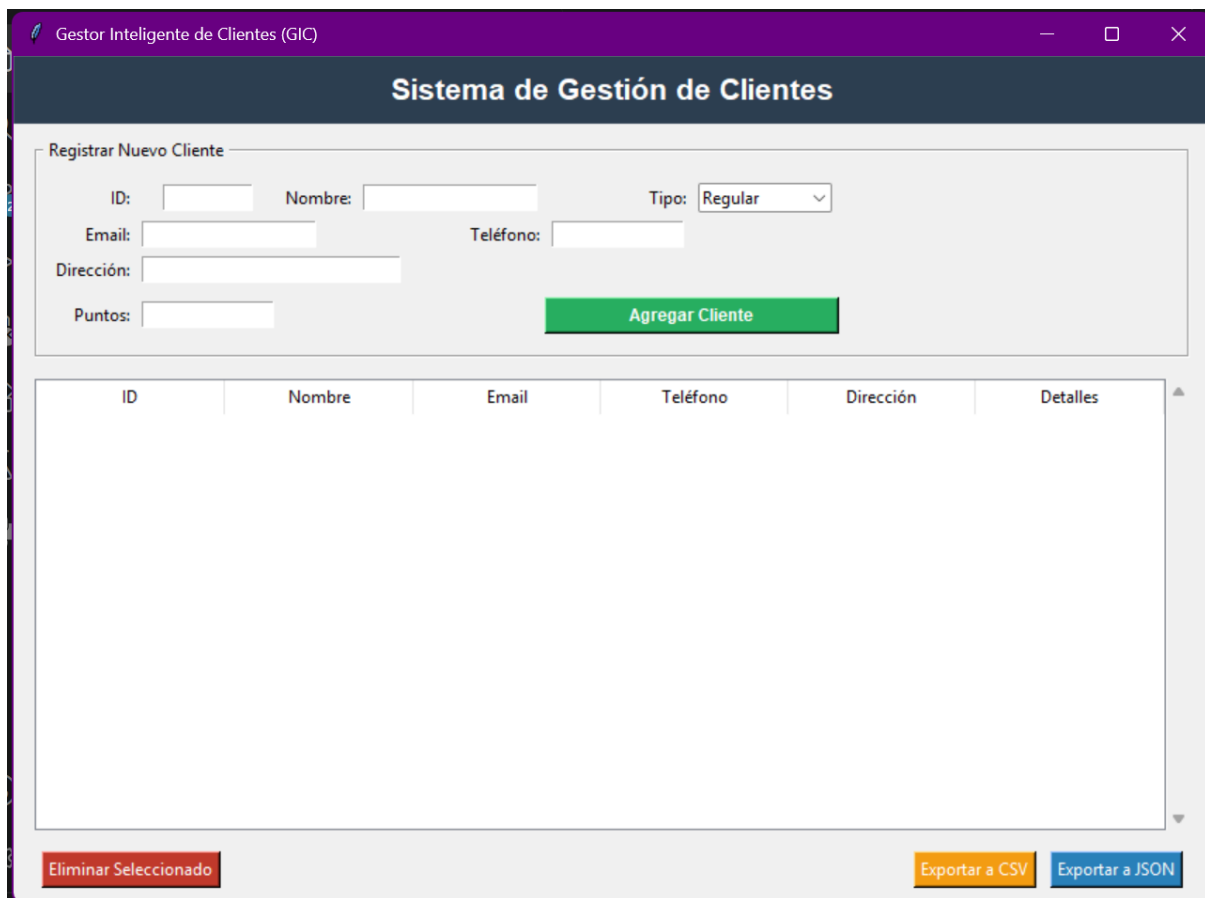
```

Figura 1. Test PASSED (OK)

1. Resumen Ejecutivo del Proyecto

El Gestor Inteligente de Clientes (GIC) es una solución integral orientada a objetos diseñada para optimizar la administración, clasificación y persistencia de información de clientes en pequeñas y medianas empresas.

El software permite registrar, modificar, buscar y eliminar registros de clientes clasificados por categorías, garantizando la consistencia de datos mediante persistencia relacional en SQLite, auditoría en archivos planos y comunicación asíncrona simulada mediante APIs REST externas.



Gestor Inteligente de Clientes (GIC)

Sistema de Gestión de Clientes

Registrar Nuevo Cliente

ID: Nombre: Tipo:

Email: Teléfono:

Dirección:

Puntos:

ID	Nombre	Email	Teléfono	Dirección	Detalles
----	--------	-------	----------	-----------	----------

Figura 2. Inicio GIC

2. Arquitectura de Software y Módulos

El sistema fue construido siguiendo el principio de responsabilidad única (SRP) y una separación limpia en componentes modulares:

Módulo/Archivo	Responsabilidad principal
<u>cliente.py</u>	Define la clase abstracta base Cliente y sus subclases derivadas (ClienteRegular, ClientePremium, ClienteCorporativo).
<u>gestorclientes.py</u>	Núcleo de la lógica de negocio. Gestiona la colección de clientes en memoria, coordina guardados en BD y triggers de API.
base_datos.py	Capa de datos (DAO). Administra la conexión relacional SQLite (clientes.db), creación de tablas y consultas CRUD.
<u>archivos.py</u>	Mapeador de exportación de reportes a formatos estructurados (TXT, CSV con delimitador punto y coma, y JSON).
<u>api.py</u>	Servicio de integración con servicios web HTTP (validación de dominios de correo y notificaciones automatizadas).
<u>excepciones.py</u>	Jerarquía de errores personalizados (ClienteError, ClienteYaExisteError, EmailInvalidoError, etc.).
<u>interfaz.py</u> y <u>main.py</u>	Capa de presentación gráfica desarrollada en Tkinter para interacción amigable con el usuario final.
test/	Suite de pruebas unitarias automatizadas con la librería nativa unittest (test_cliente.py y test_gestor.py).

3. Fundamentos de Programación Orientada a Objetos (POO)

El diseño del software implementa rigurosamente los pilares fundamentales de la POO:

1. Encapsulamiento: Se protegen las variables de instancia (como identificador, email, teléfono) restringiendo su acceso directo y exponiéndose a través de métodos accesorios getters.
2. Herencia: La clase abstracta Cliente centraliza las propiedades comunes. De ella heredan tres clases especializadas:

 ClienteRegular: Incorpora el atributo puntos_acumulados.

 ClientePremium: Incorpora nivel_membresia y porcentaje_descuento.

 ClienteCorporativo: Incorpora empresa y limite_credito.

3. Polimorfismo: Cada tipo de cliente redefine (sobrescribe) el método obtener_datos() para dar un formato único y personalizado a la información generada.
4. Uso de super(): Permite reutilizar el constructor de la clase padre Cliente dentro de los inicializadores de las clases hijas.

4. Integración con APIs REST y Persistencia

Para enriquecer el flujo de trabajo, el método crearCliente se conecta mediante peticiones HTTP GET y POST a endpoints públicos:

1. Validación de Correo: Antes de registrar un nuevo usuario, ServicioExternoAPI.validar_email_api(email) verifica que la dirección posea la estructura requerida.
2. Notificación Automática: Al guardar exitosamente al cliente en SQLite, se dispara un evento HTTP POST a ServicioExternoAPI.enviar_email_bienvenida para simular el envío de una confirmación.

5. Verificación mediante Pruebas Unitarias

Resultado de la Ejecución de Tests:

Se implementaron 8 casos de prueba automatizados cubriendo instancias, cálculos de descuento, lanzamiento de excepciones ante duplicados y persistencia.

6. Conclusión

El proyecto cumple cabalmente con todos los requerimientos técnicos fijados en la pauta de evaluación. La combinación de POO, arquitectura desacoplada, pruebas unitarias y persistencia de datos garantiza un código robusto, escalable y listo para un entorno de producción.